Structure de données : les arbres

A.MAZZA

MP-3

13 octobre 2025



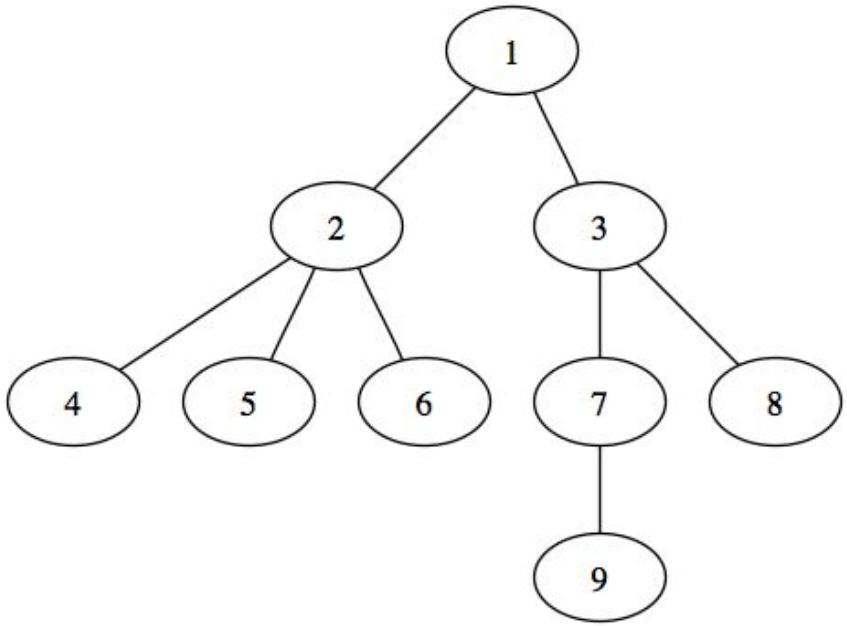
Plan

1. Généralités sur les arbres
   - Définition
   - Terminologie et propriètés
   - Applications
   - Mesures sur les arbres
2. Les arbres binaires
   - Définition des arbres binaires
   - Implantation d'un arbre binaire
   - Les primitives (fonctions) sur les arbres binaires
   - Notion de parcours
3. Les arbres binaires de recherche
4. Les arbres binaires complets
5. Requête dans un ABR
   - Recherche
   - Successeur et prédécesseur
   - Insertion et suppression
6. Exercices :

Définition

- Un arbre (tree en anglais) est une structure de données non **linéaire**.
- Un arbre = ensemble de sommets ou noeuds tel que :

  ∃ un sommet unique appelé **racine** qui na pas de supérieur (père).
  Tous les autres sommets sont atteints à partir de la racine par une branche unique.
- Représentattion graphique d'un arbre :



Terminologie et propriètés

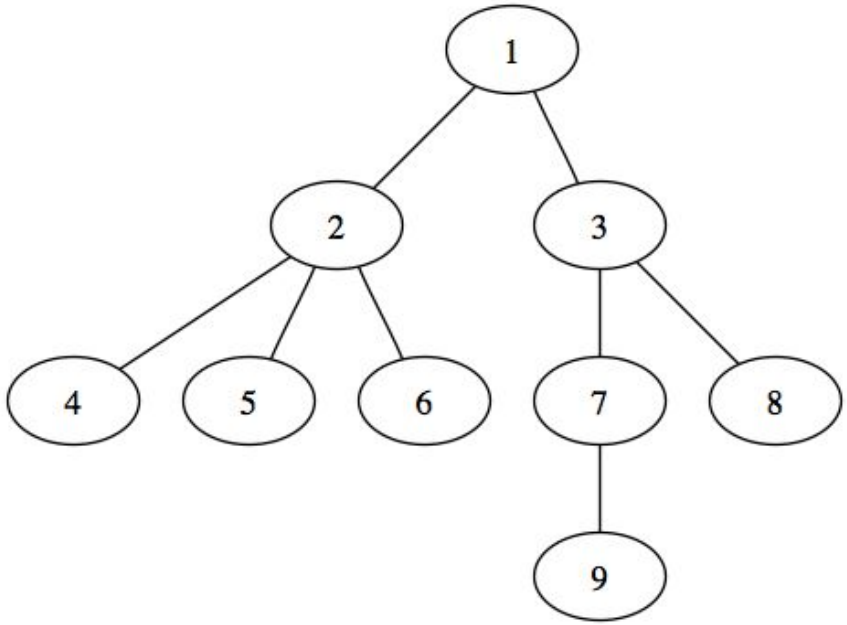
- **noeud** = sommet
- **racine**= noeud sans père
- Une **feuille** est un sommet qui na pas de fils.
- **branche** : chemin entre 2 noeuds
- **père dun sommet** : le prédécesseur dun sommet
- **fils dun sommet** : les successeurs dun sommet
- **frères** : des sommets qui ont le même père
- Tout sommet x qui nest pas la racine a :

  un unique parent, noté parent(x) (appelé père parfois)
  0 ou plusieurs fils. fils(x) désigne lensemble des fils de x
- Si x et y sont des sommets tels que x soit sur le chemin de r à y alors

  x est un ancêtre de y
  y est un descendant de x

Terminologie et propriètés

⇒Exemple :



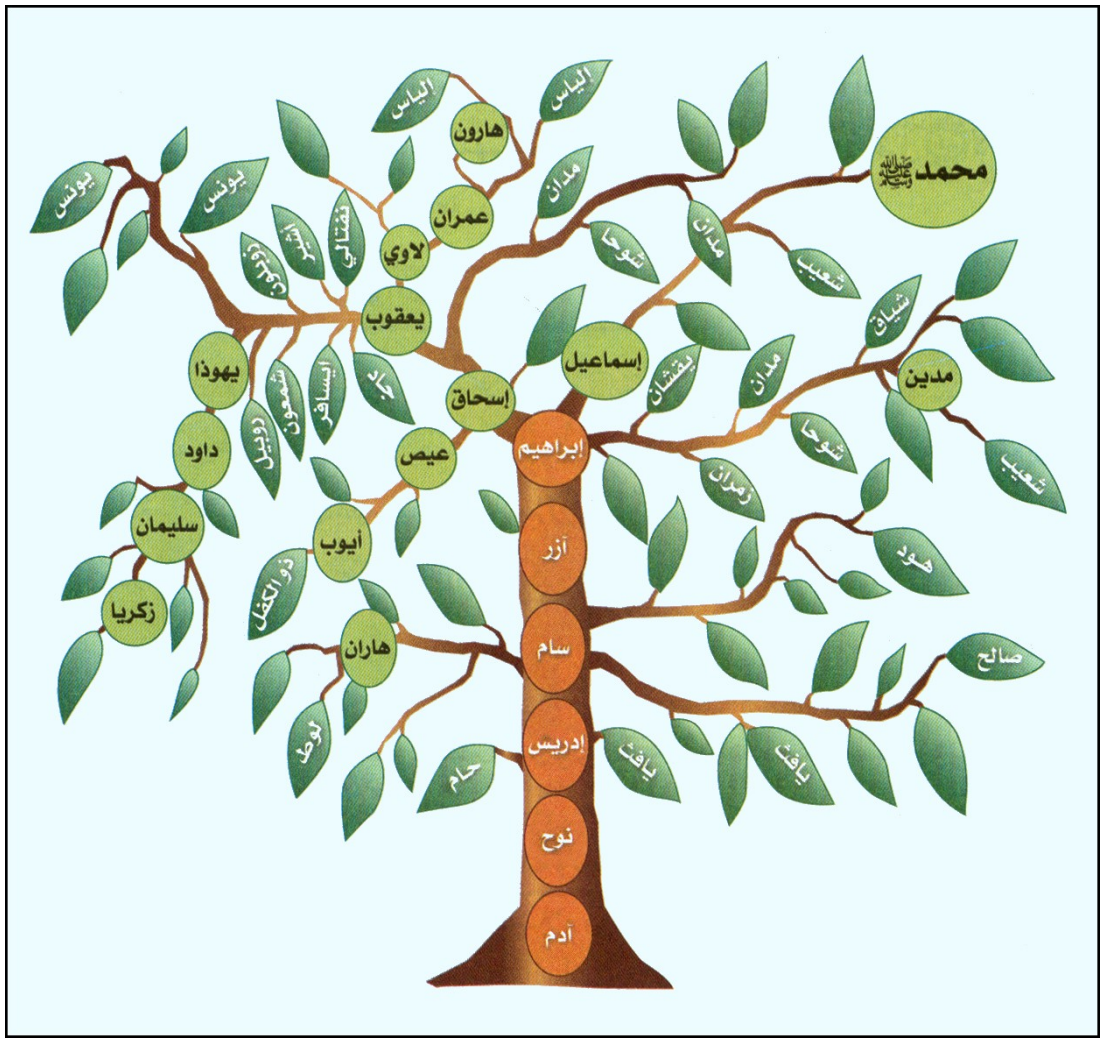
- 1 est la racine
- 4,5,6,9,8 sont les feuilles
- 9 est un descendant de 3, mais pas de 2
- 3 est un ancêtre de 9.

Applications

- Modèle pour les structures hiérachisées :
  - Arbre familial
  - Arbre généalogique
  - Structure de fichiers sous linux ou windows
  - Organigramme dune entreprise
- Les expressions arithmétiques
- En linguistique et en compilation (arbres syntaxiques).
- Gérer des bases de données
- Utilisation pour des algorithmes de recherches rapides et efficaces (tri par tas)
- Algorithmique du texte (Huffamn, prefix-trees) : compression, recherche de motifs, détection de répétitions, ...
- Intelligence artificielle : arbres de décision
- Cryptoghraphie : algorithmes de recherche

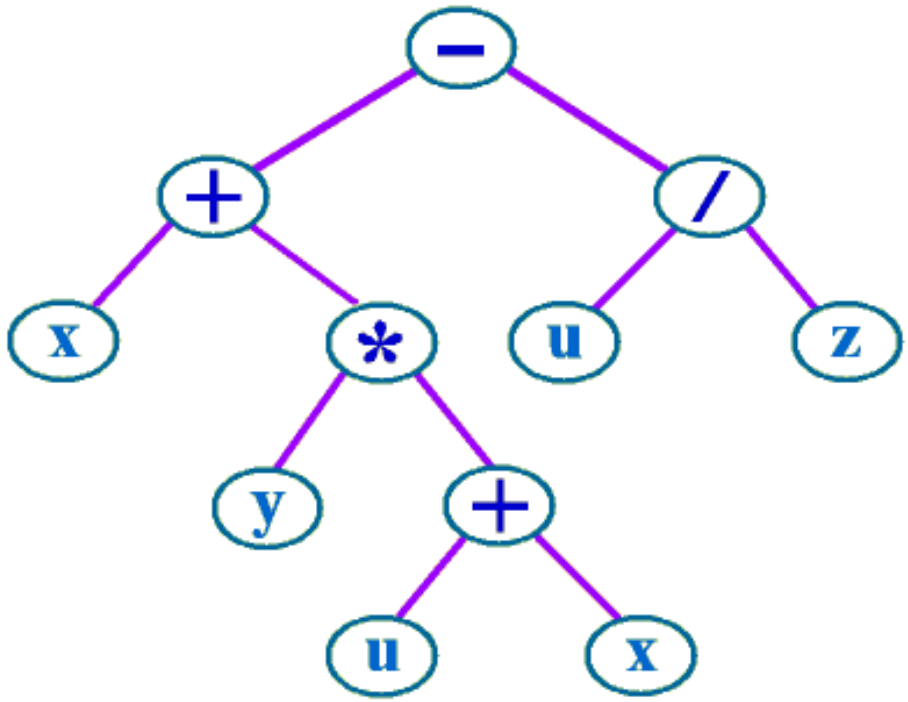
Applications

⇒ Exemple 1 : Arbre généalogique



Applications

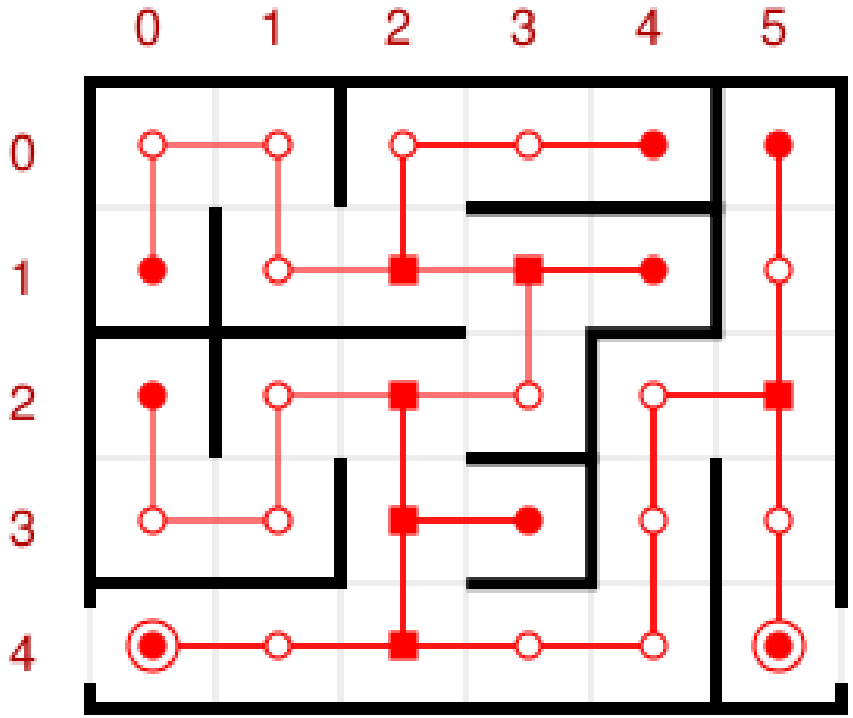
⇒ **Exemple 2 : Evaluation d'une expression algébrique**
L'évaluation de l'expression arithmétique $x + y(u + x) - u/z$ est représentée par l'arbre ci-dessous :



⇒ **Exercice :** Dessinez larbre correspondant à lexpression $3(2x - 1) + 1$ et l'arbre de l'expression $(y/2 - t)x(75 + z)$.

⇒ Exemple 3 : Intelligence artificielle : arbres de décision

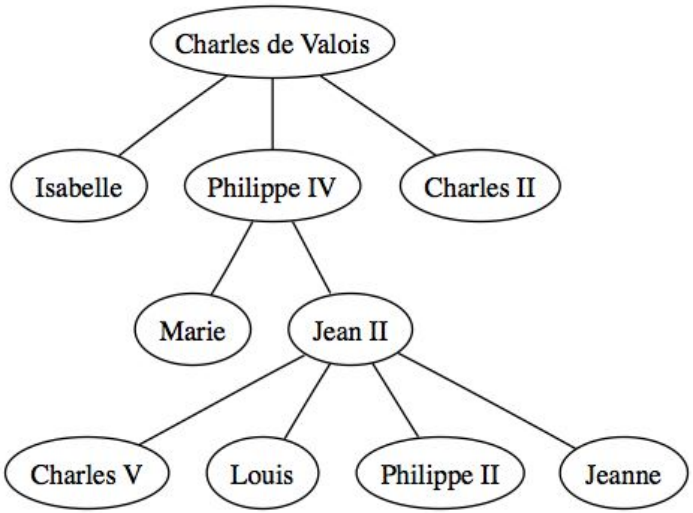
Modélisation et Création dun Labyrinthe Rectangulaire en Deux Dimensions



Arité d'un arbre

- L'arité d'un arbre représente le nombre maximum de fils qu'il possède pour un sommet donné.
- Un arbre dont les noeuds ne comporteront qu'au maximum n fils sera d'arité n. On parlera alors d'arbre n-aire.
- Il existe un cas particulièrement utilisé : c'est **l'arbre binaire**.

$\Rightarrow$ **Exemple :** Voici un arbre généalogique dont l'arité est 4



Mesures sur les arbres

La profondeur dun sommet

- La profondeur (niveau) dun noeud est la longueur de la branche depuis la racine
- La profondeur dun sommet est définie récursivement par :

  prof(v) = 0 si v est la racine
  prof(v) = prof(parent(v)) + 1

La hauteur dun sommet

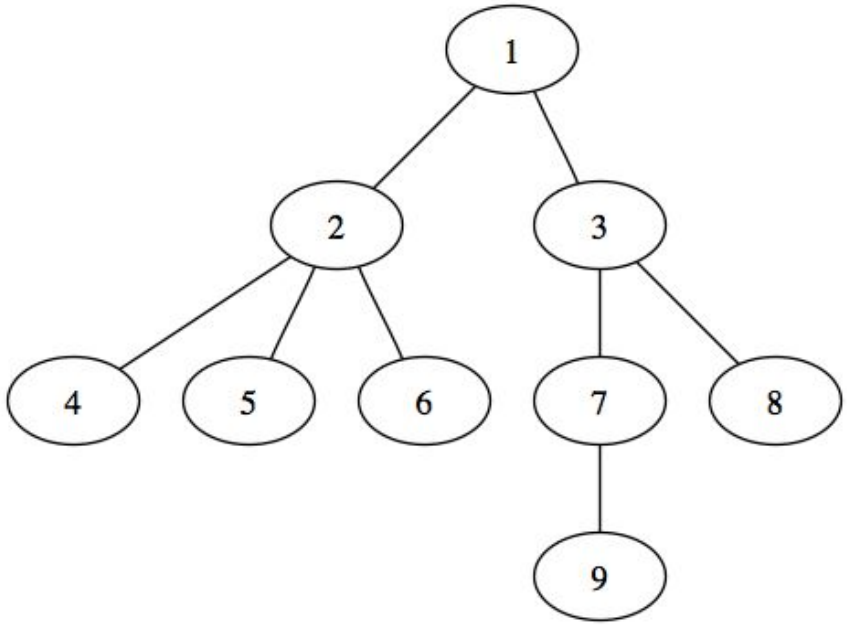
- La hauteur d'un sommet est la longueur du chemin le plus long qui part de ce sommet jusqu'à une feuille (un nud sans enfants). Autrement dit, c'est le nombre d'arêtes sur le chemin le plus long menant du nud à une feuille.
  - Les feuilles (nuds sans enfants) ont une hauteur de 0.
  - La hauteur d'un nud interne est égale à 1 plus la hauteur de son enfant le plus profond.

Autres mesures

- La hauteur dun arbre est la hauteur de sa racine
- La taille dun arbre est le nombre de ses sommets
- Le degré d'un noeud est le nombre de fils que possède ce noeud.

Mesures sur les arbres

⇒Exemple :



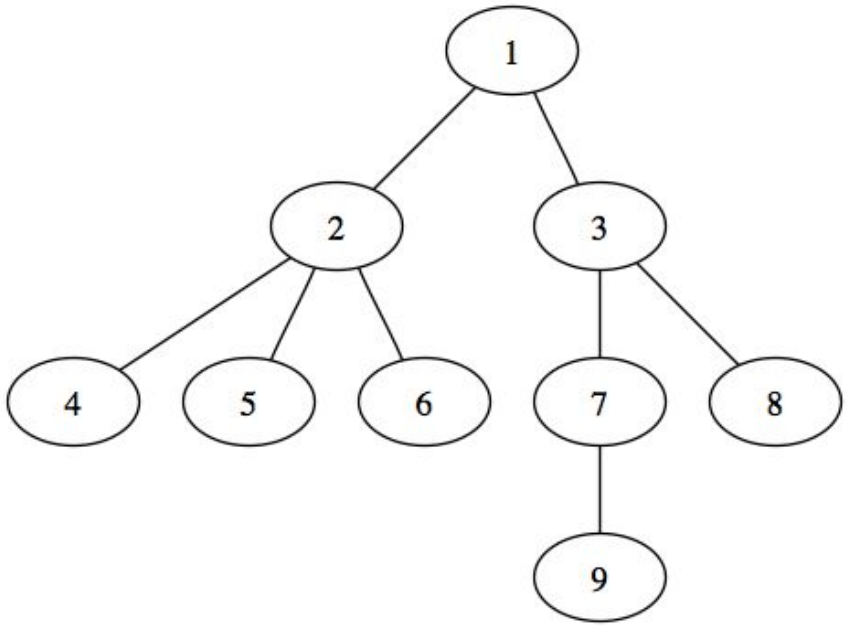
- 2,3 sont à la profondeur 1
- 4,5,6,7,8 sont à la profondeur 2
- La hauteur de 3 est 2, celle de 6 est 0, celle de 8 est 0, celle de 1 est 3
- La hauteur de l'arbre est 3
- La taille de l'arbre est 9

Représentation d'un arbre avec une liste

- On peut représenter un arbre en Python à l'aide d'une liste formée :

  dune donnée (la racine)
  suivie (éventuellement) dune suite de listes construites selon le même principe,
  qui représentent les arbres-fils de la racine.

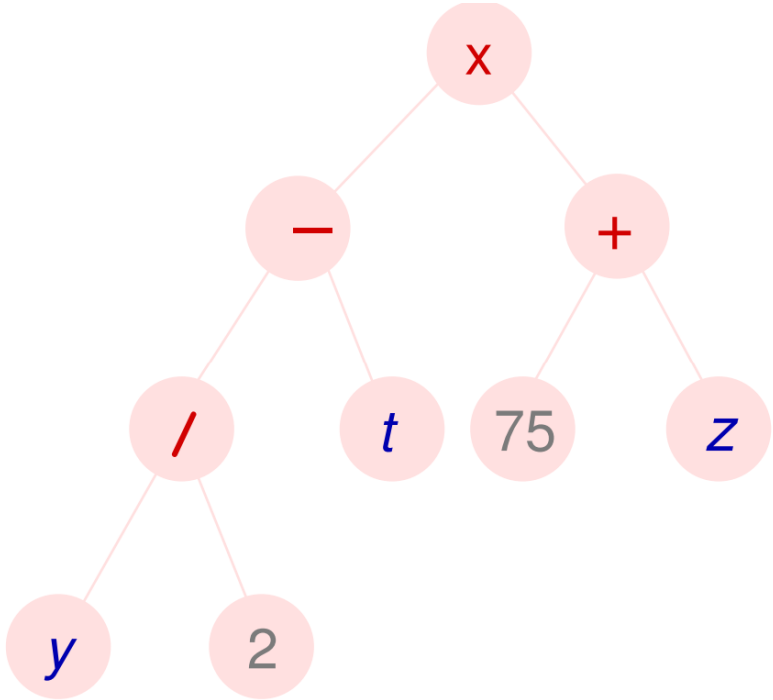
- Par exemple, larbre :



- sera représenté par la liste :$[1,[2,[4],[5],[6]],[3,[7,[9]],[8]]]$

Exemples :

Soit l'arbre suivant :



1. Donner Prof(t)= ? , Prof(y)= ?, Prof(arbre)= ?
2. Donner H(-)= ?, H(z)= ?, H(y)= ?
3. Donner une représentation de cet arbre par une liste ?

Définition des arbres binaires

Définition informelle

- Dans un arbre binaire tout noeud (sommet) a au plus deux fils
- Un arbre binaire possède exactement deux sous-arbres (éventuellement vides)

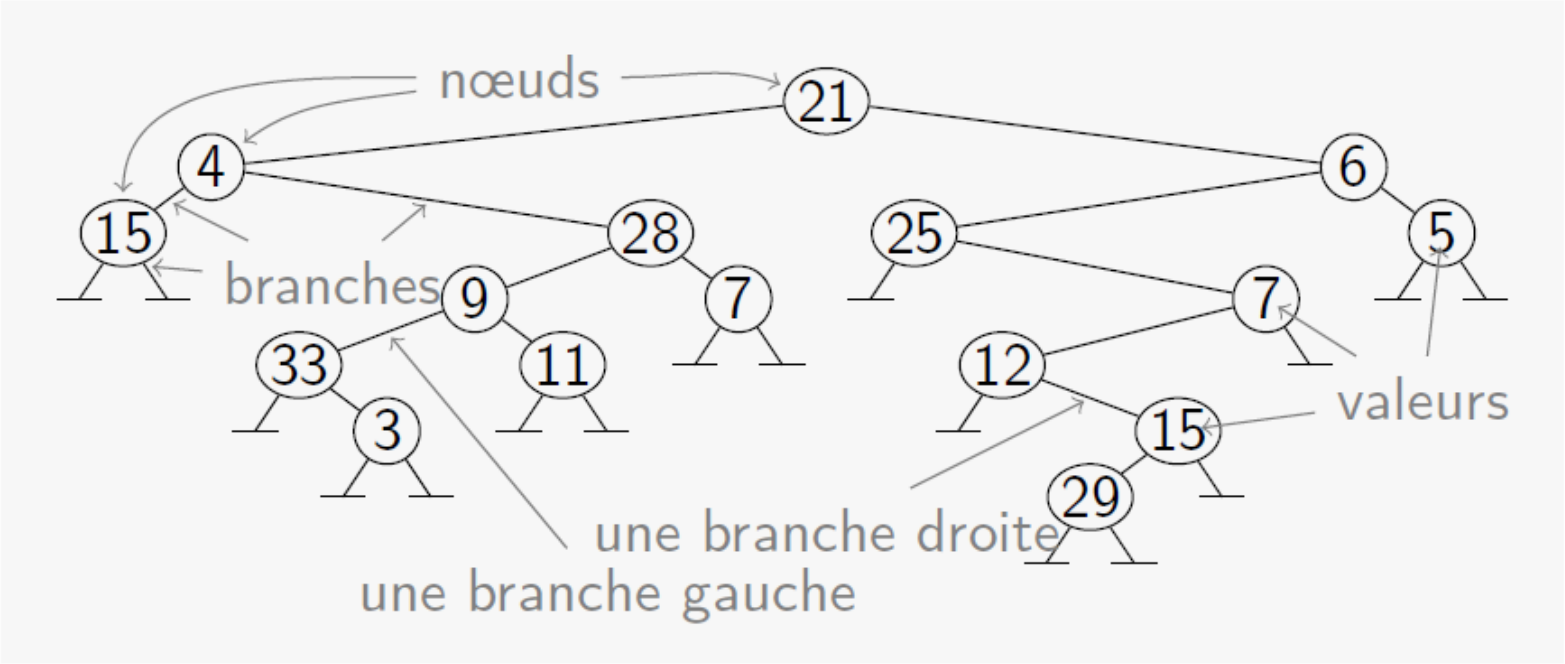
Définition récursive

- Soit vide
- Soit composé

  dune racine r
  de 2 sous arbres binaires ABG et ABD disjoints
  - ABG : sous Arbre Binaire Gauche
  - ABD : sous Arbre Binaire Droit

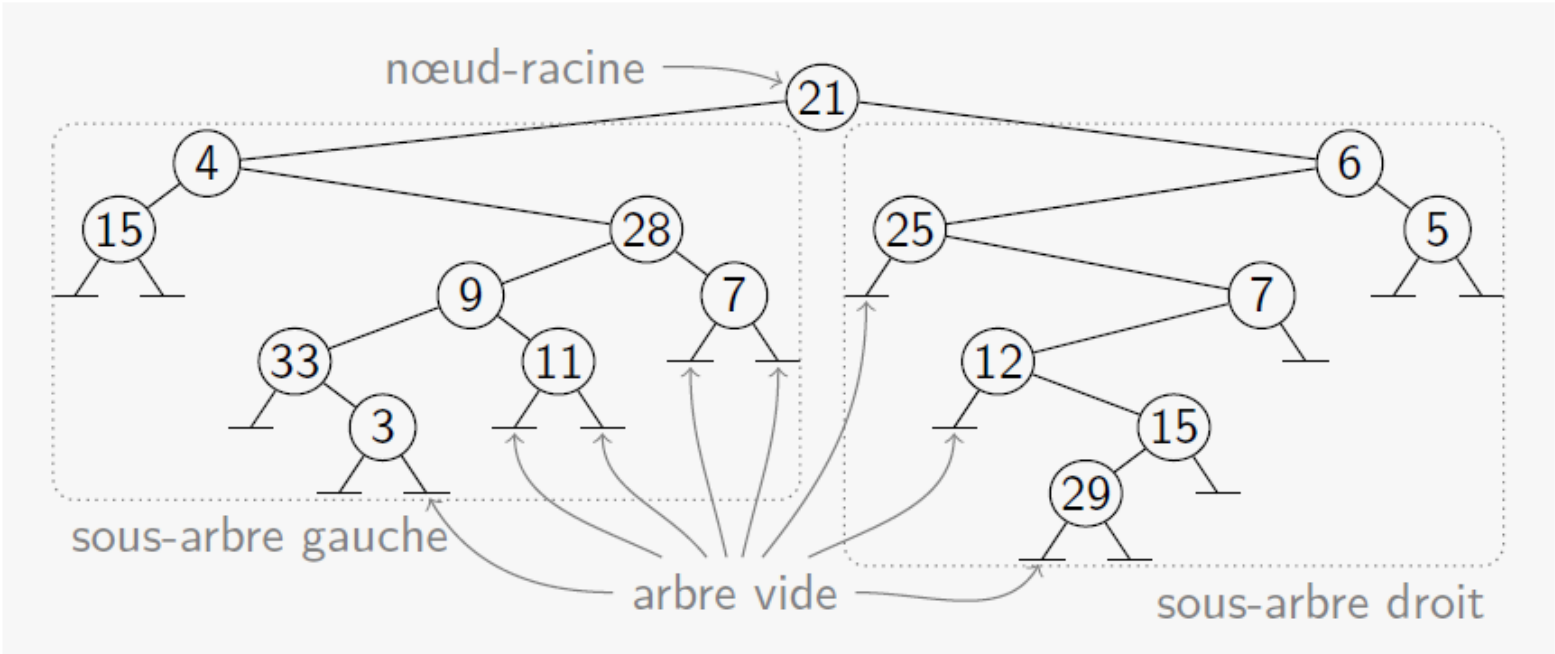
Représentation graphique d'un arbre binaire

⇒Définition informelle



Représentation graphique d'un arbre binaire

⇒Définition récursive



Représentation des arbres binaires à l'aide de listes

Définition d'un arbre binaire avec une liste

- Un **arbre** binaire est soit :

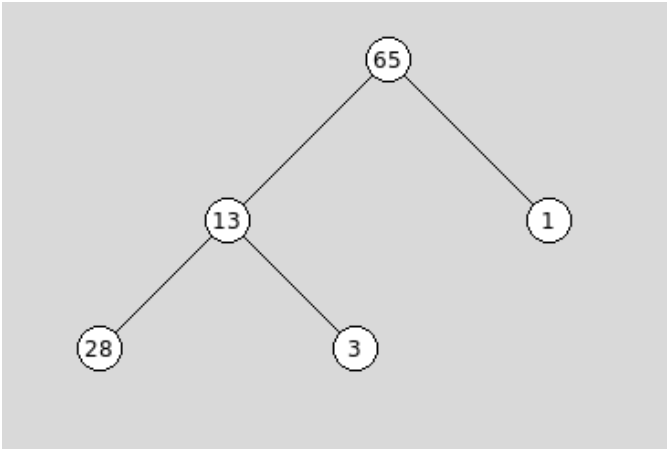
  une liste vide (arbre vide).
  Une suite de listes (ensemble de noeuds), telque, chaque liste représente un **noeud** d'un arbre binaire.
  Dans un arbre binaire non vide, une liste représente une structure de tois champs :

  1. Une étiquette (valeur)
  2. le sous-arbre gauche (ABG) (est une liste)
  3. le sous-arbre droit (ABD) (est une liste)

- La construction du sous-arbre gauche et sous-arbre droit se fait selon le même principe que la racine.

Représentation des arbres binaires à l'aide de listes

$\Rightarrow$ Exemple



Code Python

```
>>>T=[65,[13,[28,[],[]],[3,[],[]]],[1,[],[]]]
```

Représentation des arbres binaires à l'aide de dictionnaires

Définition d'un arbre binaire avec un dictionnaire

- Un **arbre** binaire est soit :

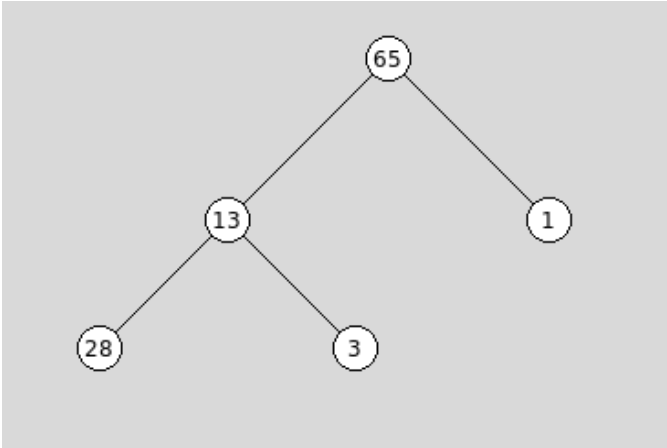
  un dictionnaire vide (arbre vide).
  Une suite de dictionnaires (ensemble de noeuds), telque, chaque dictionnaire représente un **noeud** d'un arbre binaire.
  Dans un arbre binaire non vide, un dictionnaire représente une structure de tois clés qui correspond aux trois valeurs :

  1. La clé **'rac'** avec sa valeur une étiquette
  2. La clé **'g'** avec comme valeur : le sous-arbre gauche (ABG) (est un dictionnaire)
  3. La clé **'d'** avec comme valeur le sous-arbre droit (ABD) (est un dictionnaire)

- La construction du sous-arbre gauche et sous-arbre droit se fait selon le même principe que la racine.

Représentation des arbres binaires à l'aide d'un dictionnaire

$\Rightarrow$ Exemple



Code Python

```
>>>T={'rac' : 65,' g' : {'rac' : 13,' g' : {},' d' : {}},
'd' : {'rac' : 1,' g' : {'rac' : 28,' g' : {'rac' : 3,' g' : {},
'd' : {}},' d' : {}},' d' : {}}}
```

Créer un arbre vide **def ArbreVide()**

Principe

- La fonction **ArbreVide()** permet de créer un arbre vide,
- elle renvoie une liste vide [ ]

- **Code Python**

Créer un arbre vide **def ArbreVide()**

Principe

- La fonction **ArbreVide()** permet de créer un arbre vide,
- elle renvoie une liste vide [ ]

- **Code Python**

```
def ArbreVide() :
        return []
```

Tester si l'arbre binaire est vide def EstVide(A)

Principe de la fonction

- La fonction **EstVide()** teste si l'arbre est vide ou non,
- elle renvoie 1 si elle est vide, et 0 sinon.

- **Code Python**

Tester si l'arbre binaire est vide def EstVide(A)

Principe de la fonction

- La fonction **EstVide()** teste si l'arbre est vide ou non,
- elle renvoie 1 si elle est vide, et 0 sinon.

- **Code Python**

```
def EstVide(A) :
        if (A==[ ]) :
                return True
        else :
                return False
```

Valeur de la racine : def ValRacine(A)

Principe de la fonction

- La fonction **ValRacine** renvoie l'élément à la racine de l'arbre A s'il n'est pas vide.
- Elle retourne None si l'arbre A est vide.

- **Code Python**

Valeur de la racine : def ValRacine(A)

Principe de la fonction

- La fonction **ValRacine** renvoie l'élément à la racine de l'arbre A s'il n'est pas vide.
- Elle retourne None si l'arbre A est vide.

- **Code Python**

```
def ValRacine(A) :
    if (A==[ ]) :
        return None
    else :
        return A[0]
```

Créer un noeud : def CreerNoeud(e,l,r)

Principe de la fonction

- La fonction **CreerNoeud** renvoie un arbre binaire dont la racine est e, le fils gauche est l et le fils droit r.
- La liste crée contient 3 cases n[0]=e, n[1]=l et n[2]=r .

- **Code Python**

  ???

Créer un noeud : def CreerNoeud(e,l,r)

Principe de la fonction

- La fonction **CreerNoeud** renvoie un arbre binaire dont la racine est e, le fils gauche est l et le fils droit r.
- La liste crée contient 3 cases n[0]=e, n[1]=l et n[2]=r .

- **Code Python**

```
def CreerNoeud(e,l,r) :
    n=[]
    n.append(e) # la valeur de la racine
    n.append(l)#fils gauche est l
    n.append(r)#fils droit est r
    return n
```

Créer une feuille : def CreerFeuille(e)

Principe de la fonction

- La fonction **CreerFeuille** renvoie un arbre binaire dont la racine est e et les fils gauche et droit sont vides.

- **Code Python : solutions** ???

Créer une feuille : def CreerFeuille(e)

Principe de la fonction

- La fonction **CreerFeuille** renvoie un arbre binaire dont la racine est e et les fils gauche et droit sont vides.

- **Code Python : solution 1**

```
def CreerFeuille(e) :
    n=[]
    n.append(e)
    n.append([])
    n.append([])
    return n
```

- **Code Python : solution 2**

```
def CreerFeuille(e) :
    return CreerNoeud(e,ArbreVide(),ArbreVide())
```

Retourner le sous arbre gauche et sous arbre droit

Principe de la fonction def FilsGauche(A)

- La fonction **FilsGauche** renvoie le fils gauche de A s'il n'est pas vide.

- **Code Python**

Principe de la fonction def FilsDroit(A)

- La fonction **FilsDroit** renvoie le fils droit de A s'il n'est pas vide.

- **Code Python**

Retourner le sous arbre gauche et sous arbre droit

Principe de la fonction def FilsGauche(A)

- La fonction **FilsGauche** renvoie le fils gauche de A s'il n'est pas vide.

- **Code Python**

```
def FilsGauche(A) :
        return A[1]
```

Principe de la fonction def FilsDroit(A)

- La fonction **FilsDroit** renvoie le fils droit de A s'il n'est pas vide.

- **Code Python**

```
def FilsDroit(A) :
        return A[2]
```

Tester si un noeud est une feuille : def EstFeuille(F)

Principe de la fonction

- La fonction **EstFeuille()** teste si un noeud est une feuille
- elle renvoie 1 si elle est une feuille, et 0 sinon.

- **Code Python**

Tester si un noeud est une feuille : def EstFeuille(F)

Principe de la fonction

- La fonction **EstFeuille()** teste si un noeud est une feuille
- elle renvoie 1 si elle est une feuille, et 0 sinon.

- **Code Python**

```
def EstFeuille(A) :
    if EstVide(A) :
        return False
    else :
        return EstVide(gauche(A)) and EstVide(droit(A))
```

Insérer une feuille comme fils droit def insertDroit(n,e)

Principe de la fonction

- La fonction **insertDroit()** insère une feuille contenant e comme fils droit d'un noeud n.
- elle provoque une erreur si n est vide ou si son fils droit n'est pas vide.

- **Code Python**

Insérer une feuille comme fils droit def insertDroit(n,e)

Principe de la fonction

- La fonction **insertDroit()** insère une feuille contenant e comme fils droit d'un noeud n.
- elle provoque une erreur si n est vide ou si son fils droit n'est pas vide.

- **Code Python**

```
def insertDroit(n,e) :
    if(not EstVide(n) and EstVide(FilsDroit(n))) :
        n[2]=CreerFeuille(e)
    else :
        print("Impossible d'insérer un noeud comme fils droit !!")
```

Insérer une feuille comme fils gauche def insertGauche(n,e)

Principe de la fonction

- La fonction **insertGauche()** insère une feuille contenant e comme fils gauche d'un noeud n.
- elle provoque une erreur si n est vide ou si son fils gauche n'est pas vide.

- **Code Python**

Insérer une feuille comme fils gauche def insertGauche(n,e)

Principe de la fonction

- La fonction **insertGauche()** insère une feuille contenant e comme fils gauche d'un noeud n.
- elle provoque une erreur si n est vide ou si son fils gauche n'est pas vide.

- **Code Python**

```
def insertGauche(n,e) :
    if(not EstVide(n) and EstVide(FilsGauche(n))) :
        n[1]=CreerFeuille(e)
    else :
        print("Impossible d'insérer un noeud comme fils gauche !!")
```

Supprimer un fils droit d'un noeud def SupprimerDroit(n)

Définition de la fonction

- La fonction **SupprimerDroit()** supprime et renvoie la racine du fils droit de n si c'est une feuille.
- Provoque une erreur si n est vide ou si son fils droit n'est pas une feuille.

- **Code Python**

Supprimer un fils droit d'un noeud def SupprimerDroit(n)

Définition de la fonction

- La fonction **SupprimerDroit()** supprime et renvoie la racine du fils droit de n si c'est une feuille.
- Provoque une erreur si n est vide ou si son fils droit n'est pas une feuille.

- **Code Python**

```
def SupprimerDroit(n) :
    if(EstVide(n) or not EstFeuille(FilsDroit(n))) :
        printf("Impossible de supprimer le fils droit !!")
    else :
        res=ValRacine(n[2])
        n[2]=[]
        return res
```

Supprimer un fils gauche d'un noeud def SuppGauche(n)

Définition de la fonction

- La fonction **SupprimerGauche()** supprime et renvoie la racine du fils gauche de n si c'est une feuille.
- Provoque une erreur si n est vide ou si son fils gauche n'est pas une feuille.

- **Code Python**

Supprimer un fils gauche d'un noeud def SuppGauche(n)

Définition de la fonction

- La fonction **SupprimerGauche()** supprime et renvoie la racine du fils gauche de n si c'est une feuille.
- Provoque une erreur si n est vide ou si son fils gauche n'est pas une feuille.

- **Code Python**

```
def SupprimerGauche(n) :
    if(EstVide(n) or not EstFeuille(FilsGauche(n))) :
        printf("Impossible de supprimer le fils gauche !!")
    else :
        res=ValRacine(n[1])
        n[1]=[]
        return res
```

Rechercher un élément : def Rechercher(A,v)

Principe de la fonction

- La fonction **Rechercher(A,v)** teste si la valeur v existe dans l'arbre A.
- elle renvoie 1 si v est un noeud de A, ou 0 sinon.

- **Code Python**

Rechercher un élément : def Rechercher(A,v)

Principe de la fonction

- La fonction **Rechercher(A,v)** teste si la valeur v existe dans l'arbre A.
- elle renvoie 1 si v est un noeud de A, ou 0 sinon.

- **Code Python**

```
def Recherche(A,v) :
    if(EstVide(A)) :
        return 0
    if(A[0]==v) :
        return 1
    if(Recherche(A[1],v)) :
        return 1
    else :
        return Recherche(A[2],v)
```

Notion de parcours

Défintion

- Un **parcours** est un algorithme qui appelle une **fonction**, sur tous les noeuds (ou les sous arbres) dun arbre.
- Lordre sur les nuds dans lequel la procédure est appelée doit être fixé.
- Il y a de nombreux choix possibles qui sont classés en deux familles :
  1. Les algroithmes de parcours en **largeur**.
  2. Les algroithmes de parcours en **profondeur**.

Quelques exemples de fonctions

- affichage,
- liste des valeurs,
- modification
- etc.

Parcours en profondeur

Défintion

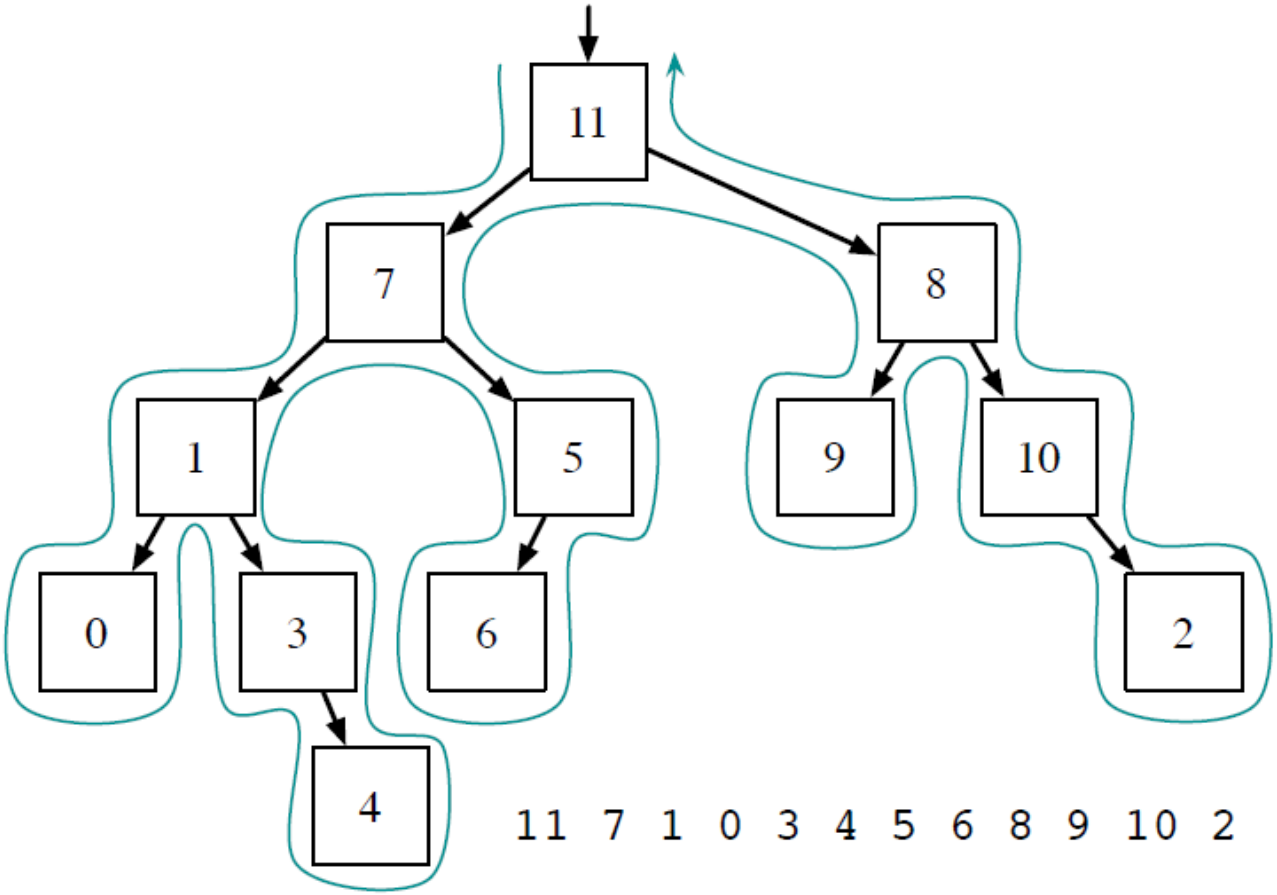
Un **parcours** est dit en **profondeur** lorsque, systématiquement, si larbre nest pas vide, le parcours de lun des deux sous-arbres est terminé avant que ne commence celui de lautre.

Les différents types

- On distingue trois types :
  1. **Parcours préfixe** (Racine Gauche Droit)
     → le noeud racine est traité au premier passage avant le parcours des sous-arbres
  2. **Parcours infixe** ou symétrique (Gauche Racine Droit)
     → le noeud racine est traité au second passage après le parcours du sous-arbre gauche et avant le parcours du sous-arbre droit
  3. **Parcours postfixe** (Gauche Droit Racine)
     → le noeud racine est traité au dernier passage après le parcours des sous-arbres

Parcours préfixe RGD

⇒Exemple :



Algorithme de parcours préfixe RGD

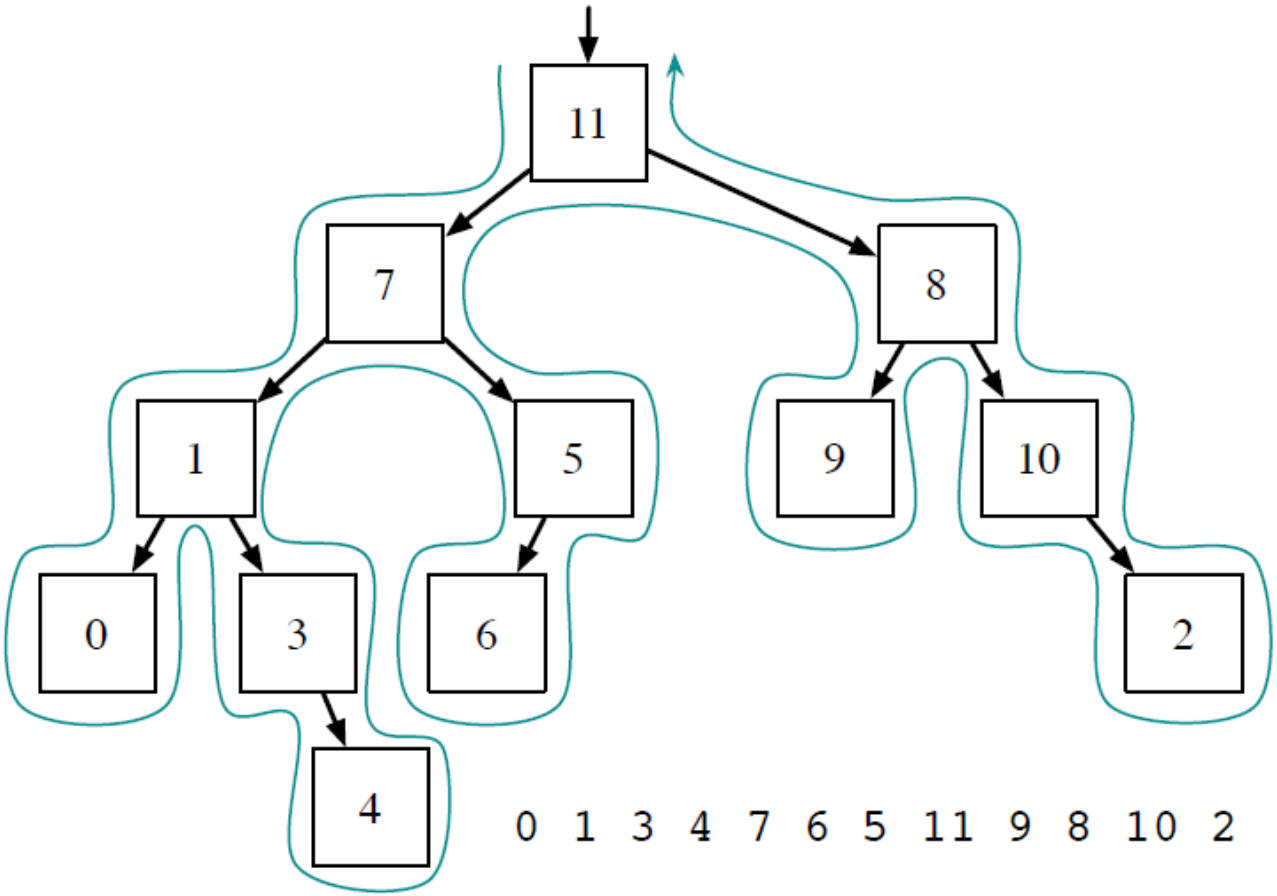
- **Principe du parcours préfixe :**
  1. application d'une fonction F à la racine,
  2. parcours préfixe du sous-arbre gauche,
  3. parcours préfixe du sous-arbre droit

- **Code Python :**

```
def ParcoursPrefixe(A) :
    if(A !=[ ]) :
        print(A[0])
        ParcoursPrefixe(A[1])
        ParcoursPrefixe(A[2])
```

Parcours infixe GRD

⇒Exemple :



Algorithme de parcours infixe GRD

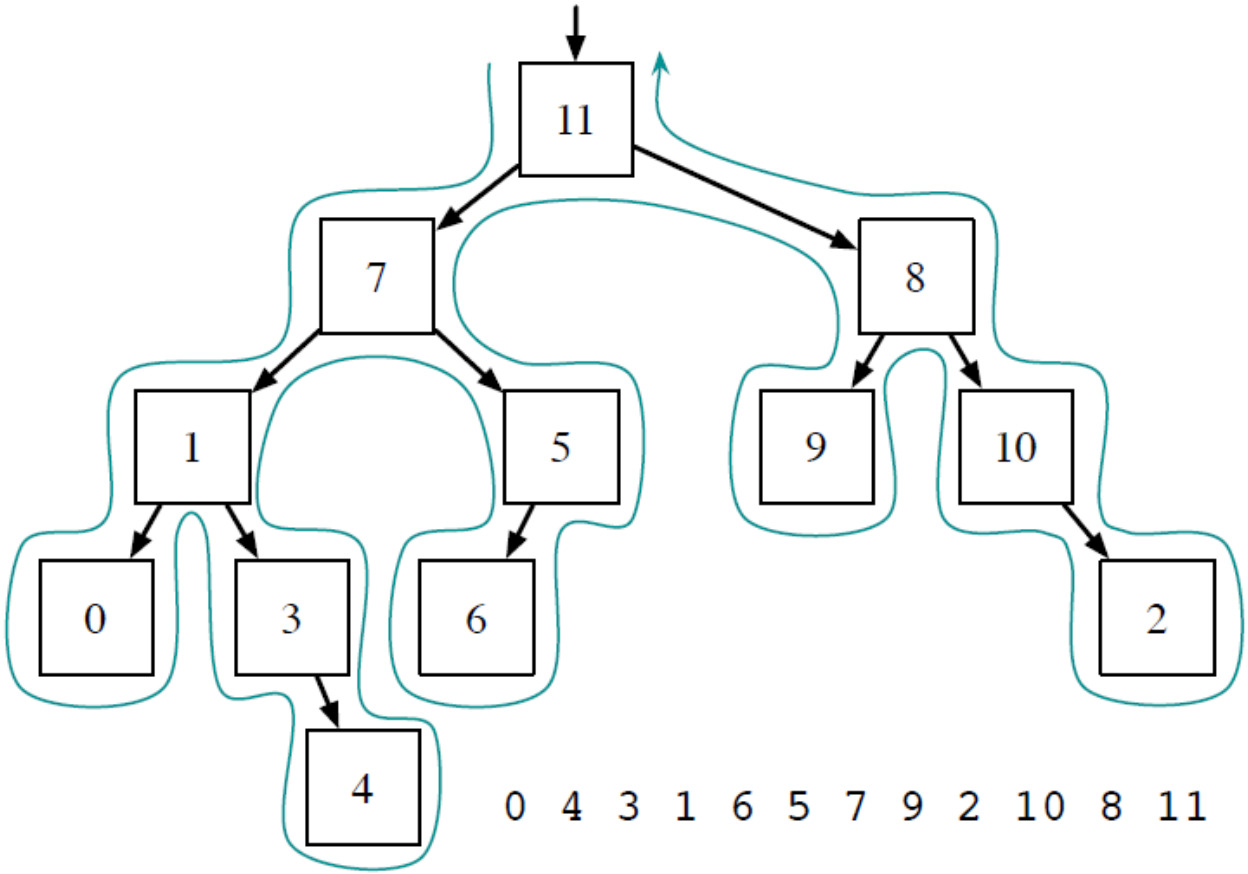
- **Principe du parcours infixe :**
  1. parcours infixe du sous-arbre gauche,
  2. application d'une fonction F à la racine,
  3. parcours infixe du sous-arbre droit

- **Code Python**

```
def ParcoursInfixe(A) :
    if(A !=[]) :
        ParcoursInfixe(A[1])
        print(A[0])
        ParcoursInfixe(A[2])
```

Parcours postfixe GDR

$\Rightarrow$Exemple :



Algorithme de parcours postfixe GDR

- **Principe du parcours postfixe :**
  1. parcours postfixe du sous-arbre gauche,
  2. parcours postfixe du sous-arbre droit
  3. application d'une fonction F à la racine,

- **Code Python :**

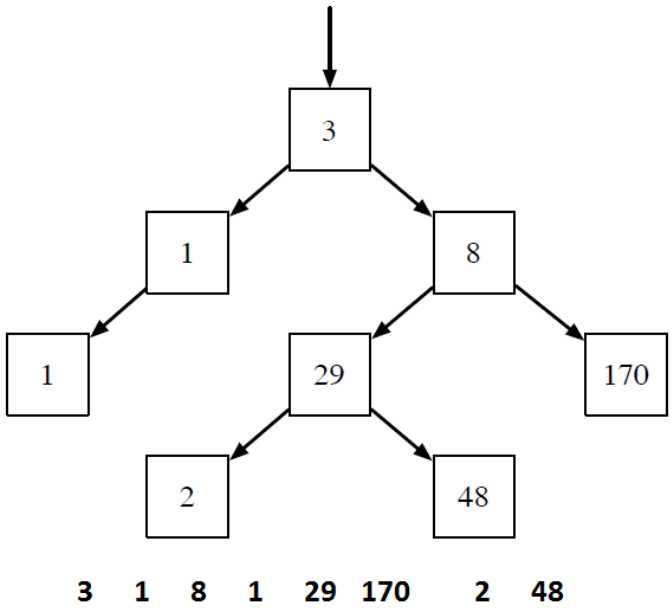
```
def ParcoursPostfixe(A) :
    if(A !=[]) :
        ParcoursPostfixe(A[1])
        ParcoursPostfixe(A[2])
        print(A[0])
```

Parcours en largeur

Définition

Un **parcours** est dit **en largeur** lorsquil procède en croissant selon les niveaux.

Voici un parcours en largeur de gauche à droite



Algorithme de parcours en largeur

Idée : on remplace la pile dappels par une file dattente dans lalgorithme de parcours préfixe

ParcoursEnLargeur (arbre A)
Début
créer une file vide F
SI A est non vide ALORS
Enfiler A dans F
Tant que (F non vide) faire
A $<$ $--$ Défiler(F)
traitement(A)
SI ABG de A non vide ALORS
Enfiler ABG de A dans F
FSI
SI ABD de A non vide ALORS
Enfiler ABD de A dans F
FSI
FinTQ
FSI
détruire F
Fin

Parcours en largeur en Pyhton

- **Code**

```
def parcoursEnLargeur(a) :
    """Retourne la liste L des noeuds en largeur avec la fameuse technique de la file"""
    #je n'utilise pas les files déjà implémentées parce que trop lourdes
    #à ajouter ici
    F = [a]
    L = [ ]
    while len(F)>0 :
        #je prends le premier élément, je l'affiche
        L.append(ValRacine(F[0]))
        #j'ajoute ses fils s'il y en a, à la fin de la file F
        if not EstVide(FilsGauche(F[0])) :
            F.append(FilsGauche(F[0]))
        if not EstVide(FilsDroit(F[0])) :
            F.append(FilsDroit(F[0]))
        #je défile
        F= F[1 :]
    return L
```

Les arbres binaires de recherche

Définition

Un **arbre binaire de recherche** (ABR) est un **arbre binaire** danslequel, tous les noeuds du sous-arbre gauche d'un noeud ont une valeur inférieure (ou égale) à la sienne et tous les noeuds du sous-arbre droit ont une valeur supérieure à la valeur du noeud lui-même.

intérêt

- les arbres de recherche ont un intérêt quand il y a un grand nombre dajouts et de suppressions.
- On peut utiliser la structure ABR dans le but de réaliser des algorithmes de tris rapides, exemple tri par tas.

Représentation d'un ABR

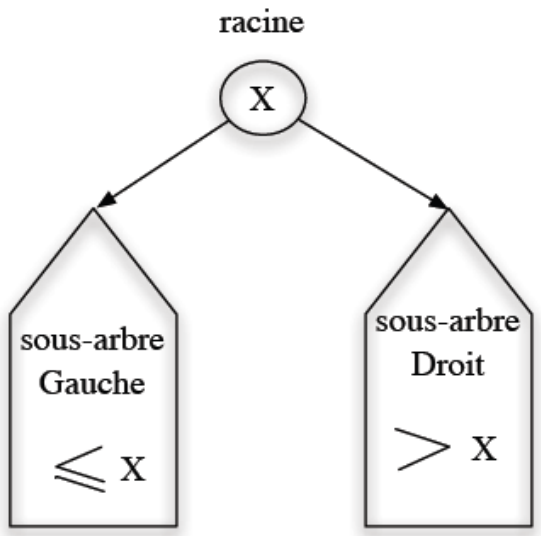
On utilise la même implémentation que les arbres binaires.

Définition récursive d'un ABR

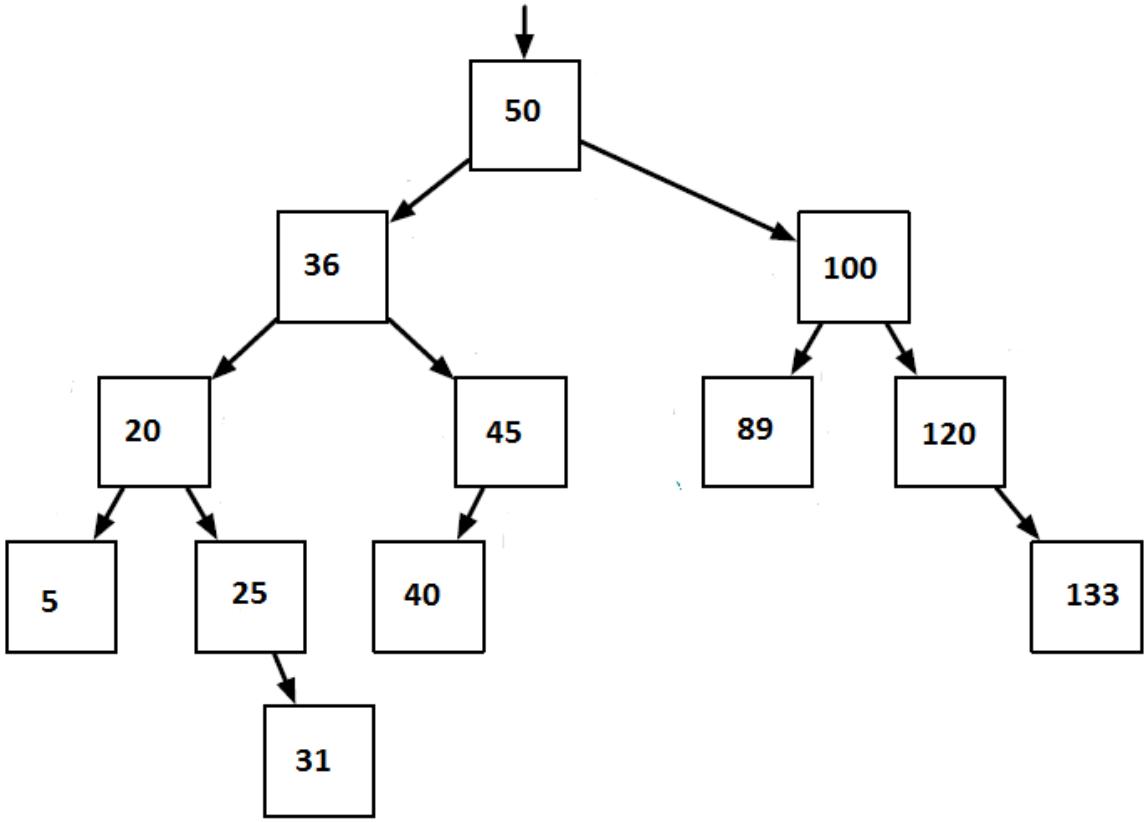
- Soit vide
- Soit composé

  un noeud racine contient un élément X

  de 2 sous arbres binaires de recherche ABRG et ABRD disjoints
  - dans ABRG $\rightarrow$ noeuds $\leq$ X
  - dans ABRD $\rightarrow$ noeuds $>$ X



⇒ **Exemple :** Représentation graphique d'un arbre binaire



Création d'un ABR

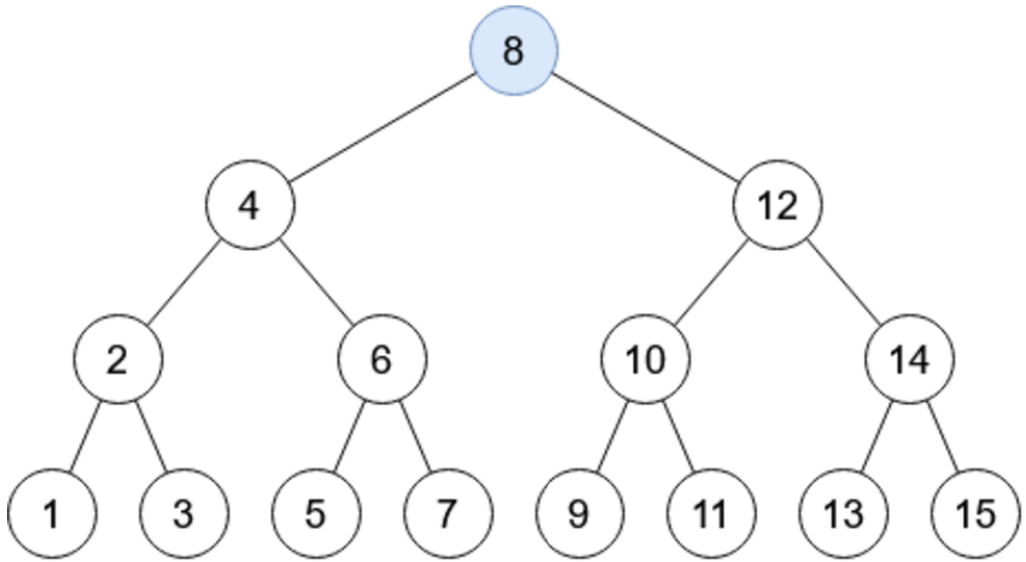
On construit un ABR à partir des insertions successives des éléments d'une liste $L = [4, 20, 12, 2, 7, 3, 6, 0, 15, 1, 13, 14]$

Les arbres binaires parfaits

Définition

Arbre binaire **parfait** : arbre dont tous les niveaux sont remplis et dont toutes les feuilles sont à la même profondeur (le dernier niveau est complètement occupé).

⇒**Exemple dun arbre binaire parfait :**

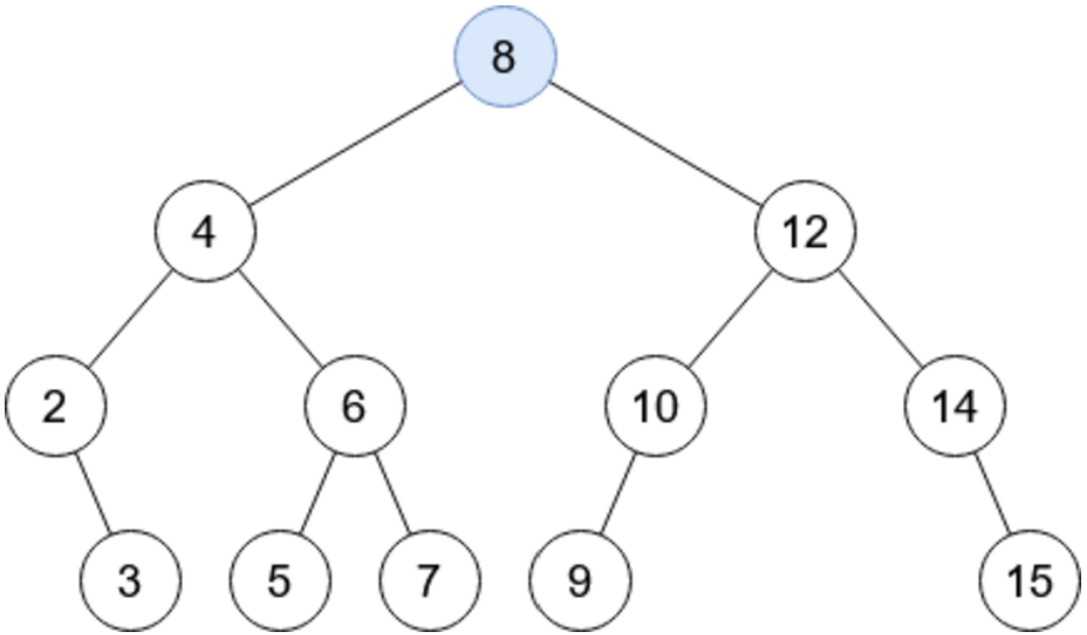


Les arbres binaires équilibrés

Définition

Arbre binaire **équilibré** : tous les chemins de la racine aux feuilles ont la même longueur.

⇒**Exemple dun arbre binaire équilibré :**

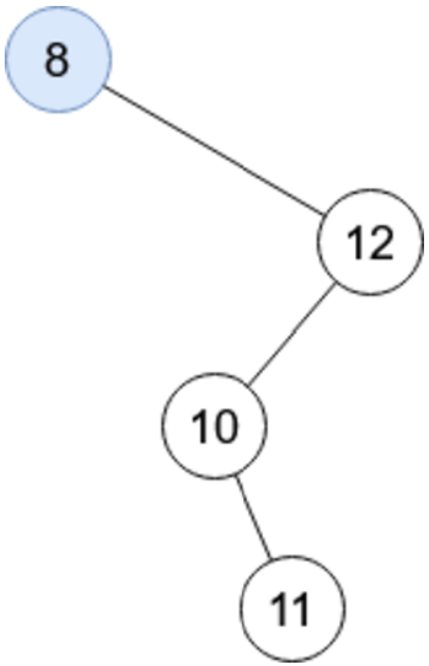


Les arbres binaires dégénérés

Définition

Arbre binaire **dégénéré** : chacun de ses nuds a au plus un fils.

⇒**Exemple dun arbre binaire dégénérés :**



Les arbres binaires complets

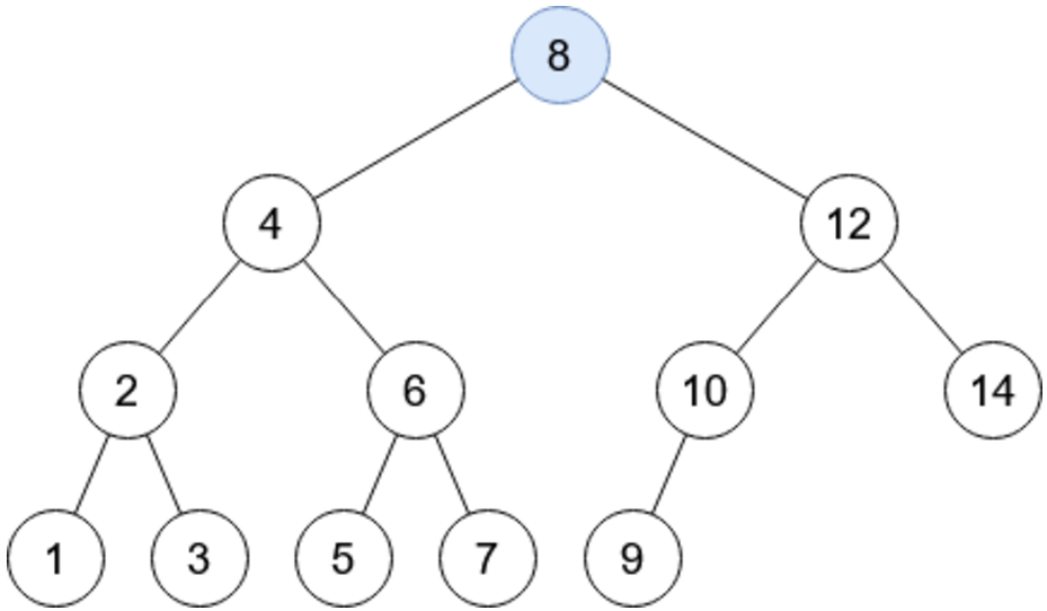
Définition

Un arbre binaire **complet** est un arbre binaire tel que :

- Si h dénote la hauteur de larbre :

  Pour tout $i \in [0, h-1]$, il y a exactement 2^i noeuds à la profondeur i .
  Une feuille a une profondeur h ou $h-1$.
  Les feuilles de profondeur maximale (h) sont tassées sur la gauche.

⇒**Exemple dun arbre binaire complet :**



Rechercher une valeur

On utilise la procédure suivante pour rechercher un nud ayant une clé donnée dans un arbre binaire de recherche. Étant donné un pointeur sur la racine de larbre et une clé k, ARBRE-RECHERCHER retourne un pointeur sur un nud de clé k sil en existe un ; sinon, elle retourne NIL

ARBRE-RECHERCHER(x, k)

1 **si** $x =$ NIL ou $k = clé[x]$
2 **alors retourner** x
3 **si** $k < clé[x]$
4 **alors retourner** ARBRE-RECHERCHER$(gauche[x], k)$
5 **sinon retourner** ARBRE-RECHERCHER$(droite[x], k)$

Rechercher minimum

On peut toujours trouver un élément dun arbre binaire de recherche dont la clé est un **minimum** en suivant les pointeurs gauches à partir de la racine jusquà ce quon rencontre `NIL`, comme le montre la figure suivante. La procédure suivante retourne un pointeur sur lélément minimal du sous-arbre enraciné au nud x.

ARBRE-MINIMUM(x)

1 **tant que** $gauche[x] \neq$ NIL
2 **faire** $x \leftarrow gauche[x]$
3 **retourner** x

Minimum et maximum

Rechercher maximum

Le pseudo-code de ARBRE-MAXIMUM est symétrique.

ARBRE-MAXIMUM(x)

1 **tant que** $droite[x] \neq$ NIL
2 **faire** $x \leftarrow droite[x]$
3 **retourner** x

Successeur

Successeur

Si toutes les clés sont distinctes, le successeur dun nud x est le nud possédant la plus petite clé supérieure à clé[x]. La structure dun arbre binaire de recherche permet de déterminer le successeur dun nud sans même effectuer de comparaison entre les clés. La procédure suivante retourne le successeur dun nud x dans un arbre binaire de recherche, sil existe, et `NIL` si x possède la plus grande clé de larbre.

ARBRE-SUCCESSEUR(x)

```
1  si droite[x] ≠ NIL
2     alors retourner ARBRE-MINIMUM(droite[x])
3  y ← p[x]
4  tant que y ≠ NIL et x = droite[y]
5       faire x ← y
6             y ← p[y]
7  retourner y
```

Insertion

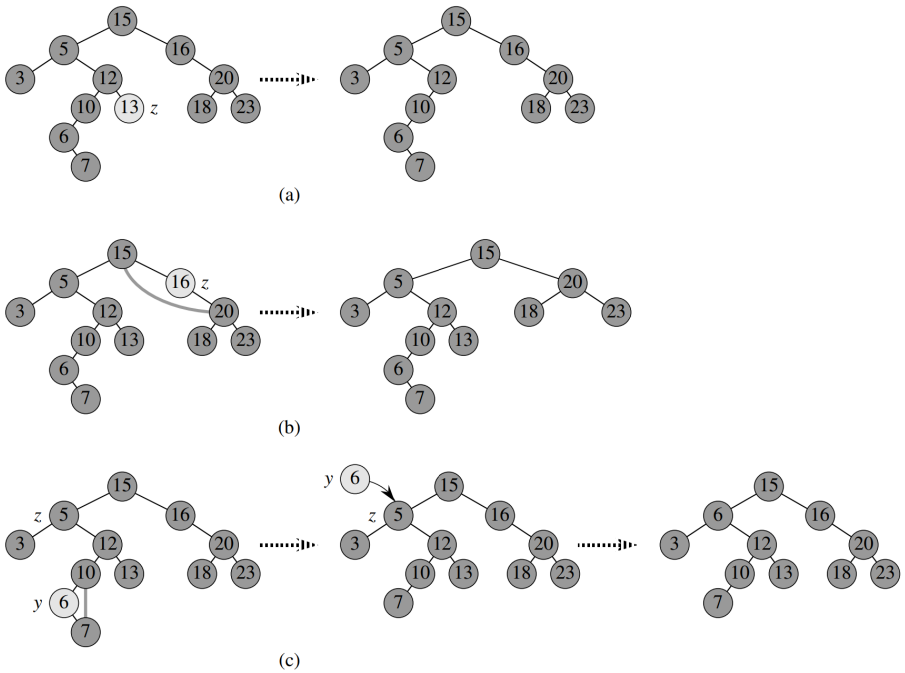
Pour insérer une nouvelle valeur v dans un arbre binaire de recherche T, on utilise la procédure ARBRE-INSÉRER. On lui passe un nud z pour lequel clé[z] = v, gauche[z] = NIL et droite[z] = NIL. Elle modifie T et certains champs de z pour permettre à z dêtre inséré à sa position correcte dans larbre.

ARBRE-INSÉRER(T, z)

```
1   y ← NIL
2   x ← racine[T]
3   tant que x ≠ NIL
4       faire y ← x
5             si clé[z] < clé[x]
6                alors x ← gauche[x]
7                sinon x ← droite[x]
8   p[z] ← y
9   si y = NIL
10     alors racine[T] ← z                ▷ arbre T était vide
11     sinon si clé[z] < clé[y]
12               alors gauche[y] ← z
13               sinon droite[y] ← z
```

Suppression

Suppression dun nud z dun arbre binaire de recherche. Pour savoir quel est le nud à supprimer effectivement, on regarde le nombre denfants de z ; ce nud est colorié en gris clair. (a) Si z na pas denfant, on se contente de le supprimer. (b) Si z na quun seul enfant, on détache z. (c) Si z a deux enfant, on détache son successeur y qui possède au plus un enfant et on remplace alors la clé et les données satellites de z par celles de y.



Pseudo-code de la suppression

```
ARBRE-SUPPRIMER(T, z)
 1  si gauche[z] = NIL ou droite[z] = NIL
 2     alors y ← z
 3     sinon y ← ARBRE-SUCCESSEUR(z)
 4  si gauche[y] ≠ NIL
 5     alors x ← gauche[y]
 6     sinon x ← droite[y]
 7  si x ≠ NIL
 8     alors p[x] ← p[y]
 9  si p[y] = NIL
10     alors racine[T] ← x
11     sinon si y = gauche[p[y]]
12              alors gauche[p[y]] ← x
13              sinon droite[p[y]] ← x
14  si y ≠ z
15     alors clé[z] ← clé[y]
16           copier données satellites de y dans z
17  retourner y
```

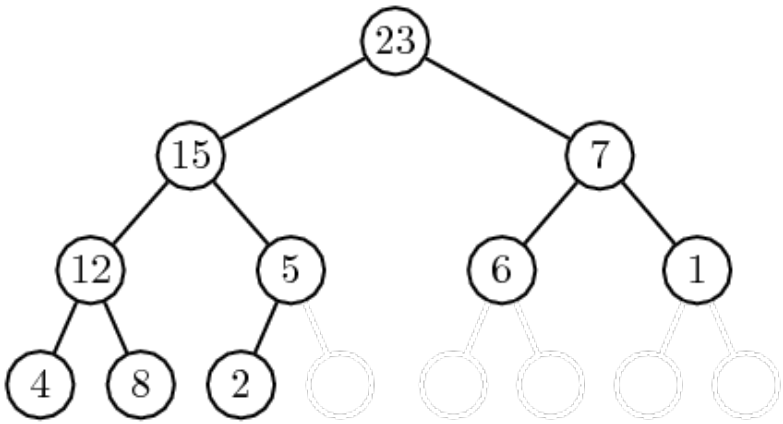
Exercices

1. On suppose que des entiers compris entre 1 et 1000 sont disposés dans un arbre binaire de recherche et que lon souhaite trouver le nombre 363. Parmi les séquences suivantes, lesquelles ne pourraient pas être la suite des nuds parcourus ?
   - 2, 252, 401, 398, 330, 344, 397, 363.
   - 924, 220, 911, 244, 898, 258, 362, 363.
   - 925, 202, 911, 240, 912, 245, 363.
   - 2, 399, 387, 219, 266, 382, 381, 278, 363.
   - 935, 278, 347, 621, 299, 392, 358, 363.
2. Écrire des versions récursives des procédures ARBRE-MINIMUM et ARBREMAXIMUM.
3. Écrire la procédure ARBRE-PRÉDÉCESSEUR.

Définition

- Un tas (heap en Anglais) est un arbre binaire complet qui vérifie en plus **la propriété suivante :** la clé de chaque noeud doit être supérieure ou égale (resp. inférieure ou égale) aux clés de chacun de ses fils.
- Dans le premier cas ($\leq$), on parle de tas-max (ou max-heap)
- Dans le second cas ($\geqslant$) on parle de tas-min (ou min-heap).

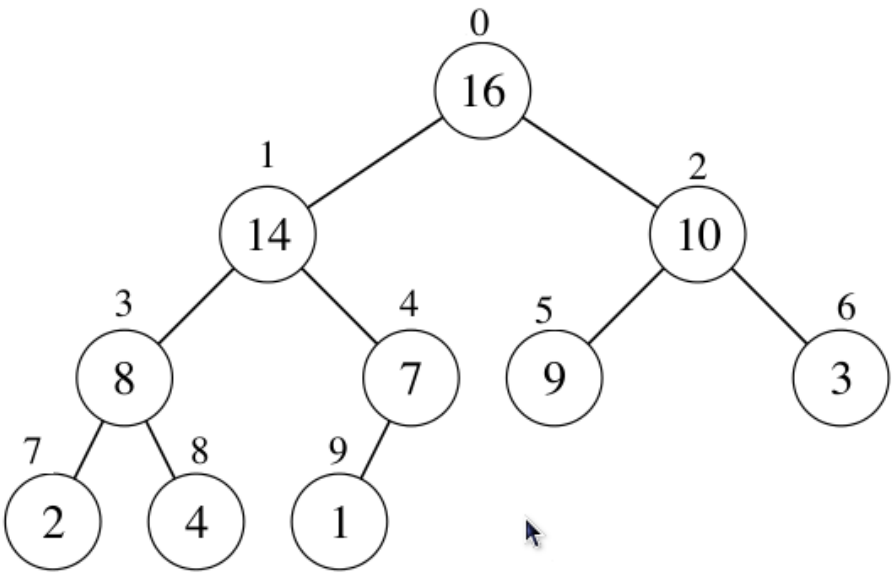
$\Rightarrow$**Exemple dun Tas :**

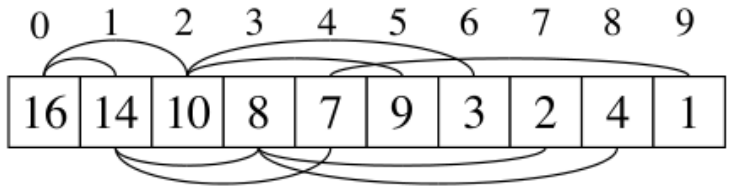


Implémentation d'un tas

On procède la même façon qu'un arbre binaire complet, c-à-d on utilise une liste d'une seule dimension de taille $2^n - 1$

⇒**Exemple**





Ajout dun élément x à un **tas-min** T

Principe de la fonction def insere_tas(T,x) :

- Mettre x dans T[n], où n est la première place disponible dans le tas.
- On compare alors cette valeur à celle contenue dans le noeud père (cest-à-dire T[n/2]) ; si elle est supérieure, le processus sarrête puisque lon a encore un tas. Sinon, on échange les éléments T[n] et T[n/2].
- On remonte alors au père, et on réitère lopération précédente jusquà ce que le tas soit convenablement ordonné.

Ajout dun élément x à un **tas-min** T

- **Code Python**

```
def insere_tas(T,x) :
    T.append(x)
    i=len(T) - 1
    pere = (i-1)//2
    # on remonte dans larbre jusquà trouver la bonne place
    while pere >= 0 and T[pere] >= x :
    #on permute le fils avec son père
        T[i],T[pere]=T[pere],T[i]
        i = pere
        pere = (i-1)//2
```

Complexité

- Si le tas a n éléments, le nombre dopérations nexcédera pas la hauteur de larbre correspondant.
- Or la hauteur dun arbre binaire complet de n noeuds est [(log2 (n))]+1.
- Donc lalgorithme dinsertion dun élément dans un tas est de complexité O(log(n)) ordonné.

Suppression de lélément racine d'un **tas-min** T

Principe de la fonction def supprime_racine_tas(T,x) :

Pour supprimer le premier élément du tas (donc le minimum dans le cas dun tas-min), il faut alors retirer la racine de larbre, ce qui donne deux arbres (fils gauche et fils droit). Le plus simple pour reformer un seul arbre est dappliquer lalgorithme suivant :

- on met lélément le plus à droite de la dernière ligne (donc le dernier élément de la liste) à la place de la racine.
- on compare alors sa valeur avec celle de ses fils et on échange cette valeur avec la plus petite de ces deux valeurs (bien sûr, il faut faire attention, quand un noeud na quun fils).
- on réitère cette opération en descendant dans larbre jusquà ce que le tas soit convenablement ordonné.

Suppression de lélément racine d'un **tas-min** T

- **Code Python**

```
def supprime_racine_tas(T) :
    x =T.pop()#pour supprimer le dernier élément de la liste T
    T[0] = x #on remplace la racine avec le dernier élément
    n = len(T) - 1
    i = 0
    while (2*i + 1) <= n :
        j = 2*i + 1#indice de fils gauche
        if j < n :
            if T[j + 1] < T[j] :#on compare le fils gauche avec le fils droit
                j =j+ 1
        if x <= T[j] :
            T[i] = x
            return()
        T[i],T[j]=T[j],T[i] #on permute le pere avec le plus petit de ses fils
        i = j
```

⇒

Lalgorithme de suppression de la racine est de complexité O(log(n)).

Principe de tri maximier

Définition

Le tri maximier (appelée aussi ńtri par tasż ou ńheapsortż) est basée sur la **notion de Tas.** Cet algorithme de tri est proposé par Williams en 1964. Le fait d'organiser l'ensemble des éléments à trier sous la forme d'un **Tas** permet d'obtenir une sélection du maximum en un temps qui est dans l'ordre de $log_2(n)$, ce qui permet à l'algorithme de redevenir optimum.

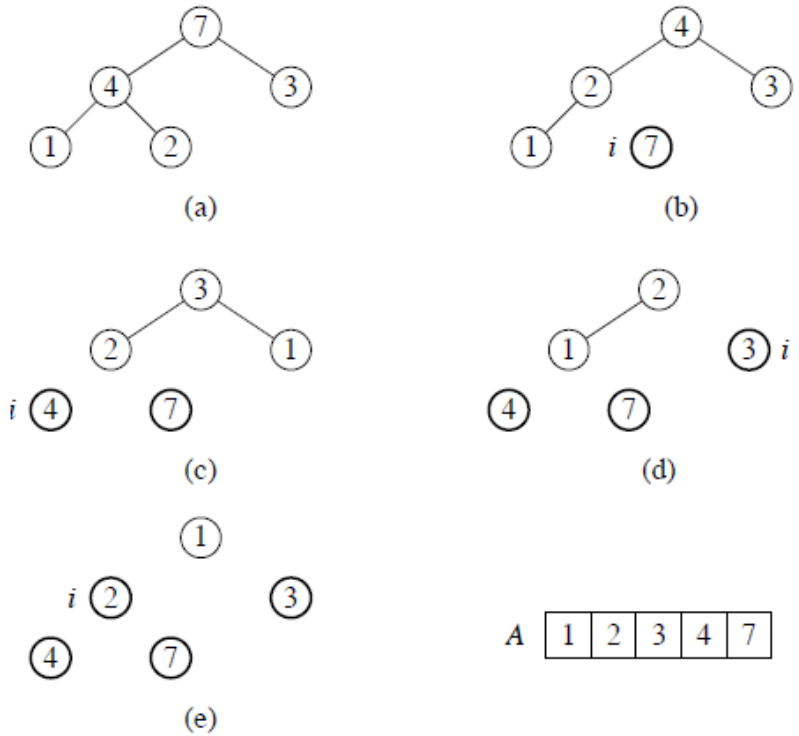
Principe de fonctionnement

- La méthode du tri par tas comporte deux étapes :
  1. On Construit par des insertions successives un tas à partir de la liste des éléments à trier.
  2. On remplace la racine, qui est nécessairement le plus grand élément du tableau par son fils le plus grand. La racine est transférée à la place de la dernière feuille de larbre et celle-ci est repositionnée. On réitère avec la nouvelle racine autant de fois quil y a de noeuds dans larbre.

Des exemples d'illustration

Exemple 1

- On va illustrer le tri maximier sur le tableau A=[7,4,3,1,2].



Implémentation de l'algorithme de tri maximier

Tri maximier

- Pour implémenter l'algorithme tri maximier on a besoin de programmer quatre fonctions :
  1. Une fonction **insere_tas(T,x)** permettant d'insérer un élément x dans la bonne position dans un tas T.
  2. Une fonction **Liste_To_tas(L)** permettant de constuire un tas à parir d'une liste L.
  3. Une fonction **supprime_racine_tas(T)** permettant de supprimer la racine d'un tas T.
  4. Une fonction **Tri_maximier(T)** permettant de trier la liste T en uitlisant l'algorithme tri par tas.

```
def Liste_To_tas(L) :
    T=[] #tas vide
    for i in L :
        insere_tas(T,i)
    return T
```

Implémentation de l'algorithme de tri maximier

Code **Tri_maximier(T)**

```
def Tri_maximier(L) :
    #on construit un tas à partir de la liste L
    tas=Liste_To_tas(L)
    #construction de la liste triée
    liste_tri = [0]*len(L)
    for i in range(len(L)) :
        liste_tri[i] = tas[0]
        # la fonction supprime_racine_tas ne fonctionne pas si le len(tas)=1
        if len(tas)>1 :
            supprime_racine_tas(tas)
    return(liste_tri)
```

⇒**Exemple :**

```
>>>L=[10,39,21,2,8,6,1]
>>>T=Tri_maximier(L)
>>>print(T)
[1, 2, 6, 8, 10, 21, 39]
```